Lösungen der Warm-up-Karten

Gleiche Nummerierung wie auf den Karten. Fachlich richtige Alternativen sind gleichwertig – entscheidend sind die genannten Kernpunkte.

W3-01 Das Echo im Brunnen · Tracetabelle

a) Die Aufrufe gehen von oben nach unten „in die Tiefe“; die Rückgabewerte entstehen in umgekehrter Reihenfolge von unten nach oben: 4 → 4 + 7 = 11 → 11 + 1 = 12 → 12 + 8 = 20. Rückgabewert von echo(4718): 20.

b) echo liefert die Quersumme von n: n % 10 ist die letzte Ziffer, n / 10 ist die Zahl ohne die letzte Ziffer.

c) Basisfall: n < 10 (einstellige Zahl, Rückgabe ohne Selbstaufruf). Rekursionsschritt: echo(n / 10) + n % 10. Die Rekursion endet, weil n / 10 für n ≥ 10 echt kleiner als n ist und nie negativ wird – nach so vielen Schritten, wie n Ziffern hat, ist der Basisfall erreicht.

```
Rekursionstiefe | Aufruf     | Rückgabewert
----------------+------------+-------------
       0        | echo(4718) |     20
       1        | echo(471)  |     12
       2        | echo(47)   |     11
       3        | echo(4)    |      4
```

W3-02 Die Hydra des Labyrinths · Funktionsweise erläutern

a) Rückgabewert von koepfe(4): 5. Jeder Aufruf mit n ≥ 2 verzweigt in zwei Aufrufe (Baumrekursion), Blätter sind die Aufrufe mit n = 1 oder n = 0 (Rückgabe 1).

b) koepfe(4): 9 Aufrufe · koepfe(5): 15 Aufrufe · koepfe(2) wird bei koepfe(5) dreimal berechnet – dieselbe Teilaufgabe wird mehrfach gelöst.

c) A(n) = A(n − 1) + A(n − 2) + 1 (die beiden Teilbäume plus der Aufruf selbst): 1, 1, 3, 5, 9, 15, 25, … Wächst n um 1, steigt die Anzahl etwa auf das 1,6-Fache (25 / 15 ≈ 1,67; Tendenz zum goldenen Schnitt ≈ 1,618). Es gilt sogar A(n) = 2 · koepfe(n) − 1. koepfe(40) braucht damit 331 160 281 Aufrufe – über 300 Millionen. Abhilfe: iterativ rechnen oder Zwischenergebnisse speichern.

```
koepfe(4)                                  Rückgabe 5
├── koepfe(3)                              Rückgabe 3
│   ├── koepfe(2)                          Rückgabe 2
│   │   ├── koepfe(1) → 1
│   │   └── koepfe(0) → 1
│   └── koepfe(1) → 1
└── koepfe(2)                              Rückgabe 2
    ├── koepfe(1) → 1
    └── koepfe(0) → 1
```

W3-03 Der endlose Abstieg · Fehler im Java-Code

a) summeBis: Der Basisfall fehlt. summeBis(3) ruft summeBis(2), (1), (0), (−1), … auf – ohne Ende → StackOverflowError.

potenz: Der Rekursionsschritt verkleinert das Problem nicht (exponent statt exponent − 1). potenz(2, 3) ruft immer wieder potenz(2, 3) auf → StackOverflowError.

jedeZweite: Der Basisfall wird bei ungeradem n übersprungen: 7, 5, 3, 1, −1, −3, … → StackOverflowError.

b) summeBis: kein Aufruf endet. potenz: nur exponent = 0 (z. B. potenz(2, 0) = 1). jedeZweite: jedes gerade n ≥ 0, z. B. jedeZweite(6) = 12.

c) Bedingungen: (1) Es gibt einen Basisfall ohne Selbstaufruf. (2) Jeder Rekursionsschritt verkleinert das Problem so, dass der Basisfall sicher erreicht wird.

```
public static int summeBis(int n) {
    if (n == 0) {
        return 0;
    }
    return n + summeBis(n - 1);
}

public static int potenz(int basis, int exponent) {
    if (exponent == 0) {
        return 1;
    }
    return basis * potenz(basis, exponent - 1);
}

public static int jedeZweite(int n) {
    if (n <= 0) {
        return 0;
    }
    return n + jedeZweite(n - 2);
}
```

W3-04 Zählen ohne Schleife · Methodenkopf und Signatur

a) public static int zaehleAb(String s, char c, int i) – Start in zaehle: return zaehleAb(s, c, 0); (zaehle ist jetzt nur noch eine „Hülle“ mit der vorgegebenen Signatur).

b) siehe Code. Probe: zaehle("Abrakadabra", 'a') = 4 (das große A zählt nicht mit, Vergleich mit == bei Zeichen ist erlaubt).

c) enthaeltAb(r: Reihung von Ganzzahl, x: Ganzzahl, i: Ganzzahl): Wahrheitswert. Die Schleifenvariable i wird zum Parameter, der bei jedem Selbstaufruf um 1 wächst; die Abbruchbedingung der Schleife wird zum Basisfall. Statt anzahl schrittweise zu erhöhen, werden die Rückgabewerte beim Zurückkehren aus den Aufrufen addiert – die Zwischenstände liegen auf dem Aufrufstapel.

```
public static int zaehle(String s, char c) {
    return zaehleAb(s, c, 0);
}

public static int zaehleAb(String s, char c, int i) {
    if (i == s.length()) {
        return 0;
    }
    if (s.charAt(i) == c) {
        return 1 + zaehleAb(s, c, i + 1);
    }
    return zaehleAb(s, c, i + 1);
}
```

W3-05 Koordinaten im Dungeon · Lückencode

a) feld.length = 5 (Anzahl der Zeilen) · feld[0].length = 7 (Anzahl der Spalten) · feld[1][1] = 'S' · feld[3][5] = 'Z' · feld[2][3] = '#'. feld[5][0] und feld[1][7] greifen auf nicht vorhandene Indizes zu (Zeilen 0–4, Spalten 0–6) → Laufzeitfehler ArrayIndexOutOfBoundsException.

b) (1) feld.length · (2) feld[0].length · (3) feld.length · (4) feld[z].length oder feld[0].length · (5) feld[z][s]. Achtung: In (2) nicht feld[z].length – bei ungültigem z würde schon dieser Zugriff scheitern.

c) anzahlFrei() liefert 12 (S und Z zählen nicht mit). && wertet von links nach rechts aus und bricht ab, sobald der linke Teil falsch ist (Kurzschlussauswertung). Nur so wird feld[z][s] nie mit einem Index außerhalb der Karte gelesen; in umgekehrter Reihenfolge gäbe es am Rand einen Laufzeitfehler.

```
public boolean istImFeld(int z, int s) {
    return z >= 0 && z < feld.length && s >= 0 && s < feld[0].length;
}

public int anzahlFrei() {
    int anzahl = 0;
    for (int z = 0; z < feld.length; z++) {
        for (int s = 0; s < feld[z].length; s++) {
            if (feld[z][s] == '.') {
                anzahl++;
            }
        }
    }
    return anzahl;
}
```

W3-06 Der Fallen-Sensor · Methode implementieren

a) (0|0), (0|1), (0|2), (1|0), (1|2), (2|0), (2|1), (2|2) – Zeile z − 1 bis z + 1, Spalte s − 1 bis s + 1, ohne (1|1) selbst.

Kernpunkte b): Zähler mit 0 starten · Verschiebung (0|0) ausschließen · istImFeld VOR getFeld prüfen (Kurzschluss) · Vergleich von Zeichen mit == · Rückgabe der Anzahl.

c) wandNachbarn8(1, 1) = 5 · wandNachbarn8(0, 0) = 2 (nur drei Nachbarn liegen in der Karte: (0|1) und (1|0) sind Wände, (1|1) ist S) · wandNachbarn8(2, 2) = 1 (nur (2|3)).

```
public int wandNachbarn8(int z, int s) {
    int anzahl = 0;
    for (int dz = -1; dz <= 1; dz++) {
        for (int ds = -1; ds <= 1; ds++) {
            if (!(dz == 0 && ds == 0) && istImFeld(z + dz, s + ds)
                    && getFeld(z + dz, s + ds) == '#') {
                anzahl++;
            }
        }
    }
    return anzahl;
}
```

W3-07 Den Nebel lüften · Struktogramm lesen

a) Reihenfolge (Nr. (z|s) Tiefe): 1 (1|1) 0 · 2 (0|1) 1 · 3 (0|0) 2 · 4 (1|0) 3 · 5 (1|2) 1 · 6 (1|3) 2 · 7 (0|3) 3 · 8 (2|3) 3 · 9 (2|2) 4 · 10 (2|1) 5. Größte Tiefe, in der ein Feld markiert wird: 5. Am Ende sind 8 Felder gelüftet; S und Z sind wieder sichtbar.

b) 41 Aufrufe: der Erstaufruf plus genau 4 Selbstaufrufe für jedes der 10 markierten Felder (8 × '.', S und Z): 1 + 4 · 10 = 41. Alle anderen Aufrufe (Wand, außerhalb, schon '+') enden sofort.

c) Endlosrekursion: (1|1) ruft (0|1) auf, (0|1) ruft wieder (1|1) auf, das noch nicht markiert ist, usw. → StackOverflowError. Die Markierung wirkt als „Besucht“-Merker und muss vor den Selbstaufrufen gesetzt werden. Bliebe S während der Rekursion 'S', wäre es weiter begehbar und würde von (0|1) aus erneut betreten – auch S und Z müssen deshalb vorübergehend als besucht gelten.

```
Nummer (Reihenfolge):          Endzustand:
    s:   0  1  2  3                 s:  0 1 2 3
z = 0:   3  2  #  7              z = 0:  + + # +
z = 1:   4  1  5  6              z = 1:  + S + +
z = 2:   #  10 9  8              z = 2:  # + + Z
```

W3-08 Der Weg durchs Labyrinth · Tracetabelle

a) siehe Tabelle: Von S geht es zuerst nach oben in die Sackgasse (0|1), (0|0), (1|0); diese drei Felder werden auf dem Rückweg zu 'x'. Dann führt der Weg von S nach rechts über (1|2) und (1|3); dort scheitert zuerst „oben“ (0|3) (Sackgasse 'x'), „rechts“ liegt außerhalb, „unten“ ist das Ziel (2|3) in Tiefe 3. findeWeg(1, 1) liefert wahr.

b) Endzustand siehe unten, schritte = 24 (jeder Aufruf zählt, auch die, die sofort enden: außerhalb, Wand, schon besucht). Die Felder (2|1) und (2|2) wurden nie betreten und bleiben '.'.

c) Zurücknehmen: Wenn keiner der vier Nachbarn zum Ziel führt, kehrt der Aufruf mit falsch zum Aufrufer zurück; das Feld gehört nicht mehr zum Weg ('*' wird 'x'), und der Aufrufer probiert seine nächste Richtung. 'x' statt '.': Das Feld ist bereits vollständig untersucht – von dort gibt es keinen Weg. Mit '.' würde es später über einen anderen Nachbarn erneut durchsucht (unnötige Schritte); 'x' wirkt als Besucht-Markierung.

```
Rekursionstiefe | Feld (z|s) | Zeichen am Ende
----------------+------------+----------------
       0        |   (1|1)    |      S
       1        |   (0|1)    |      x
       2        |   (0|0)    |      x
       3        |   (1|0)    |      x
       1        |   (1|2)    |      *
       2        |   (1|3)    |      *
       3        |   (0|3)    |      x

Endzustand:   x x # x
              x S * *
              # . . Z
```

W3-09 Die Schatzkammer-Suche · Tracetabelle

a) Die Rückgabewerte werden unverändert nach oben durchgereicht (Endrekursion): 6 für x = 30, −1 für x = 9.

b) Teilen: mid zerlegt den Bereich lo … hi in eine linke und eine rechte Hälfte. Herrschen: Durch den Vergleich mit r[mid] steht fest, in welcher Hälfte x nur liegen kann; nur diese wird rekursiv weiter durchsucht, die andere wird verworfen. Basisfälle: Treffer (r[mid] = x) oder leerer Bereich (lo > hi).

c) Höchstens 5 Aufrufe: Der Bereich schrumpft 10 → 5 → 2 → 1 → 0 (leer). Bei 20 Elementen höchstens 6 Aufrufe – eine Verdopplung kostet nur einen Aufruf mehr.

```
binaerRek(r, 30, 0, 9):
Tiefe | lo | hi | mid | r[mid] | Rückgabe
------+----+----+-----+--------+---------
  0   |  0 |  9 |  4  |   21   |    6
  1   |  5 |  9 |  7  |   35   |    6
  2   |  5 |  6 |  5  |   26   |    6
  3   |  6 |  6 |  6  |   30   |    6

binaerRek(r, 9, 0, 9):
Tiefe | lo | hi | mid | r[mid] | Rückgabe
------+----+----+-----+--------+---------
  0   |  0 |  9 |  4  |   21   |   -1
  1   |  0 |  3 |  1  |    8   |   -1
  2   |  2 |  3 |  2  |   12   |   -1
  3   |  2 |  1 |  -  |    -   |   -1
```

W3-10 Der Spaltzauber · Funktionsweise erläutern

a) siehe Bäume. Wegen mitte = länge / 2 (ganzzahlig) ist die linke Hälfte bei ungerader Länge die kürzere: [38, 27, 43] → [38] | [27, 43].

b) mergeSort: 13 Aufrufe (ein Aufruf je Knoten des Teilungsbaums, allgemein 2n − 1 = 13). verschmelze: 6 Aufrufe (für jeden Knoten mit Länge ≥ 2, allgemein n − 1 = 6).

c) 5 Vergleiche: 3|9 → 3 · 27|9 → 9 · 27|10 → 10 · 27|43 → 27 · 38|43 → 38; danach ist der linke Teil leer, 43 und 82 werden ohne Vergleich angehängt.

```
Teilen:
Tiefe 0:           [38, 27, 43, 3, 9, 82, 10]
Tiefe 1:    [38, 27, 43]           [3, 9, 82, 10]
Tiefe 2:  [38]  [27, 43]        [3, 9]    [82, 10]
Tiefe 3:        [27] [43]       [3] [9]   [82] [10]

Verschmelzen:
Tiefe 2:  [38]  [27, 43]        [3, 9]    [10, 82]
Tiefe 1:    [27, 38, 43]           [3, 9, 10, 82]
Tiefe 0:           [3, 9, 10, 27, 38, 43, 82]
```

W3-11 Der stärkste Golem · Code ↔ Struktogramm

Kernpunkte a): Kopf mit Signatur · Basisfall als Verzweigung mit „gib r[von] zurück“ · Zuweisungen mit ← · Aufrufe als Anweisungen · Rückgabe des größeren Wertes (auch als Verzweigung mit zwei „gib … zurück“).

b) 15 Aufrufe (8 Blätter + 7 innere Aufrufe = 2 · 8 − 1), größte Rekursionstiefe 3 (8 → 4 → 2 → 1).

c) mitte = (0 + 1 + 1) / 2 = 1: Der Aufruf maximumTH(r, von, mitte) = maximumTH(r, 0, 1) ist derselbe Aufruf wie zuvor – das Problem wird nicht kleiner → Endlosrekursion (StackOverflowError). Bei jedem Bereich mit zwei Elementen tritt das auf.

```
maximumTH(r: Reihung von Ganzzahl, von: Ganzzahl, bis: Ganzzahl): Ganzzahl
  falls von = bis
    dann: gib r[von] zurück
    sonst: –
  mitte ← (von + bis) / 2
  links ← maximumTH(r, von, mitte)
  rechts ← maximumTH(r, mitte + 1, bis)
  falls links > rechts
    dann: gib links zurück
    sonst: gib rechts zurück
```

W3-12 Die Pivot-Wache · Tracetabelle

a) Nach zerlege(r, 0, 6): [3, 2, 1, 5, 8, 7, 9], Rückgabe 3, 6 Vergleiche (i = 1 bis 6). Alle kleineren Werte stehen links vom Pivot 5, alle größeren rechts; 5 steht an seiner endgültigen Position.

b) siehe Tabelle. Insgesamt 9 Aufrufe von quickSort: die 4 aus der Tabelle und 5, die sofort enden: (0, −1), (1, 1), (3, 2), (4, 4), (6, 6). Vergleiche insgesamt: 6 + 2 + 1 + 2 = 11.

c) 21 Vergleiche = 6 + 5 + 4 + 3 + 2 + 1 = 7 · 6 / 2. Das Pivot r[lo] ist jedes Mal das kleinste Element: Ein Teil ist leer, der andere nur um 1 kürzer. Statt Halbierung entsteht eine Kette der Tiefe 6.

Tiefe	Aufruf	zerlege	r nach zerlege
0	quickSort(r, 0, 6)	3	[3, 2, 1, 5, 8, 7, 9]
1	quickSort(r, 0, 2)	2	[1, 2, 3, 5, 8, 7, 9]
2	quickSort(r, 0, 1)	0	[1, 2, 3, 5, 8, 7, 9]
1	quickSort(r, 4, 6)	5	[1, 2, 3, 5, 7, 8, 9]

W3-13 Das falsche Echo · Fehler im Struktogramm

a) Fehler 1: Im sonst-Zweig wird i nicht erhöht. Schon bei i = 0 ist r[0] = 4 nicht größer als 5, also folgt derselbe Aufruf anzahlGroesser(r, 5, 0) immer wieder → Endlosrekursion (StackOverflowError). Der Rekursionsschritt verkleinert das Problem nicht.

b) Rückgabe 1 statt 2. Fehler 2: Der Basisfall greift schon bei i = Länge − 1 und liefert 0 – das letzte Element (7) wird nie geprüft. Bei einer leeren Reihung ist Länge − 1 = −1; i = 0 erreicht diesen Wert nie, es kommt zum Zugriff auf r[0] → Laufzeitfehler (Index außerhalb).

c) Basisfall i = Länge von r (leerer Rest, gib 0 zurück) und Selbstaufruf mit i + 1 im sonst-Zweig.

```
anzahlGroesser(r: Reihung von Ganzzahl, grenze: Ganzzahl, i: Ganzzahl): Ganzzahl
  falls i = Länge von r
    dann: gib 0 zurück
    sonst: –
  falls r[i] > grenze
    dann: gib 1 + anzahlGroesser(r, grenze, i + 1) zurück
    sonst: gib anzahlGroesser(r, grenze, i + 1) zurück
```

W3-14 Das Rätsel des Turmwächters · Funktionsweise erläutern

a) siehe Tabelle; nach dem Ende ist r = [2, 4, 6, 9].

b) raetsel sortiert r[0 … n − 1] aufsteigend: Es sucht im Bereich r[0 … n − 1] den Index des größten Wertes, tauscht ihn an die letzte Position n − 1 und sortiert dann rekursiv den Rest r[0 … n − 2]. Das ist Selectionsort (Auswahl des Maximums) – die äußere Schleife ist durch die Rekursion ersetzt. Basisfall: n ≤ 1 (ein Element ist sortiert); im Rekursionsschritt wird der unsortierte Bereich um 1 kürzer.

c) n = 4: 3 + 2 + 1 = 6 Vergleiche; allgemein (n − 1) + (n − 2) + … + 1 = n(n − 1)/2. Bei Verdopplung von n etwa viermal so viele Vergleiche.

```
Tiefe | Aufruf         | Belegung von r beim Aufruf
------+----------------+---------------------------
  0   | raetsel(r, 4)  | [6, 2, 9, 4]
  1   | raetsel(r, 3)  | [6, 2, 4, 9]
  2   | raetsel(r, 2)  | [4, 2, 6, 9]
  3   | raetsel(r, 1)  | [2, 4, 6, 9]
```